

Ekspertski sistem za dijagnostiku mrežnih problema

Miloš Milić SV10/2022

Motivacija

U savremenim organizacijama, čak i mali mrežni problemi mogu dovesti do ogromnih gubitaka produktivnosti. Problem je što IT tehnička osoba često nema jasnu sekvencu šta da testira prvo, a znanja iskusnijih inženjera se teško prenose.

Cilj je da kreiranjem expert sistema omogućimo bržu dijagnostiku mrežnih problema, da prenesemo znanja IT stručnjaka u sistem od pravila, i da omogućimo čak i manje iskusnom kadru da rešava mrežne probleme brzo i efikasno.

Sistem će biti offline dostupan, lako proširljiv sa novim znanjem, i baziran na jasnoj logici rezonovanja kroz tri nivoa testiranja.

Pregled problema

Mrežni problemi se često dijagnostikuju na ad-hoc osnovu, što dovodi do nekonzistentnih rezultata i dugotrajnih vremena rešavanja. Postojeći monitoring sistemi (Nagios, Zabbix) su namenjeni kontinuiranom monitoringu, a ne dijagnostici. Specijalizovani sistemi kao Thinger.io su ograničeni na određene tipove senzora.

Nema dostupnog sistema koji bi:

- Bio specijalizovan samo za mrežnu dijagnostiku
- Imao jasnu sekvencu testiranja kroz nivo (fizički, link, network, transport...)
- Objašnjavao zašto se određeni zaključak donosi
- Bio dostupan offline
- Lako se proširio sa novim pravilima

Naš sistem će omogućiti:

- Brzu dijagnostiku kroz tri nivoa rezonovanja
- Jasnu sekvencu šta testirati
- Ponovu upotrebu dijagnostičkih šablona

Metodologija rada

Postoje dva tipa ulaza u sistem:

Ulazi od IT tehničara:

- Tip simptoma: "Nema konekcije", "Nema interneta", "Spora brzina", "Čest disconnect", "Nema pristupa delu mreže"
- Vrsta uređaja: Desktop, Laptop, Mobilni telefon, Server
- Tip konekcije: Ethernet, WiFi, VPN, Dial-up
- Rezultati testova: DA/NE odgovori na pitanja
- Kontekst: Globalan ili lokalni problem, kada je počeo, šta je prethodno

Ulazi iz sistema:

- Ping testovi (local loop, gateway, DNS)
- IP konfiguracija (ipconfig, DHCP status, IP adresa)
- DNS lookup (nslookup rezultati)
- Firewall status
- Adapter status (vidljiv, onemogućen, driver)
- Gateway dostupnost
- Routing informacije

Očekivani izlazi iz sistema:

- Glavni uzrok problema sa procenatom verovatnoće
- Mogući dodatni uzroci sa nižim procentima
- Preporučeni koraci za rešavanje
- Dodatne provere ako prvi koraci ne pomognu
- Objašnjenje logike zaključivanja

Baza znanja

NIVO 1 - SIMPTOMI (Initial Hypothesis)

Pravila prvog nivoa primaju simptome od korisnika i formiraju početnu hipotezu o tome u kom delu mreže se nalazi problem. Pravila prvog nivoa rade samo sa informacijama koje korisnik prijavljuje i još uvek nema sprovedenih testova niti potvrde.

```
when NetworkProblem(symptom == "no_connection", deviceType == "laptop",  
connectionType == "wifi")
```

```
then insert(new Hypothesis("wifi_adapter_problem"));
```

```
when NetworkProblem(symptom == "no_internet", localNetworkWorks == true,  
gatewayReachable == true)
```

```
then insert(new Hypothesis("dns_or_isp_problem"));
```

```

when NetworkProblem(symptom == "slow_speed", connectivity == true)
then insert(new Hypothesis("bandwidth_or_routing_problem"));

when NetworkProblem(symptom == "frequent_disconnect", connectionType == "wifi")
then insert(new Hypothesis("wifi_interference_or_driver"));

when NetworkProblem(symptom == "no_part_access", localNetworkWorks == true)
then insert(new Hypothesis("firewall_or_routing_restriction"));

when NetworkProblem(symptom == "no_connection", deviceType == "desktop",
connectionType == "ethernet")
then insert(new Hypothesis("physical_or_ethernet_problem"));

when NetworkProblem(symptom == "no_connection", connectionType == "vpn")
then insert(new Hypothesis("vpn_authentication_or_protocol"));

when NetworkProblem(symptom == "no_internet", localNetworkWorks == false)
then insert(new Hypothesis("ip_configuration_problem"));

```

NIVO 2 - DIJAGNOSTIČKI TESTOVI (Refined Hypothesis)

Pravila drugog nivoa kombinuju početnu hipotezu sa rezultatima konkretnih dijagnostičkih testova (ping testovi, ipconfig, provera adaptera) i template alarmima koji su generisani iz pragova. Na ovom nivou hipoteza se precizira ili odbacuje na osnovu konkretnih dokaza, a uz nju se predlaže i konkretno rešenje.

```

when Hypothesis(name == "wifi_adapter_problem") NetworkTest(name ==
"local_loop_test", success == true) NetworkTest(name == "adapter_visible", success ==
false)
then insert(new Hypothesis("wifi_adapter_disabled")); insert(new Solution("Uključi WiFi
adapter u Settings"));

when Hypothesis(name == "wifi_adapter_problem") NetworkTest(name ==
"adapter_visible", success == true) NetworkTest(name == "device_manager_error", success
== true)
then insert(new Hypothesis("wifi_driver_problem")); insert(new Solution("Ažuriraj ili
reinstaliraj WiFi driver"));

when Hypothesis(name == "dns_or_isp_problem") NetworkTest(name == "gateway_ping",
success == true) NetworkAlert(name == "DNS_SLOW")
then insert(new Hypothesis("dns_server_unreachable")); insert(new Solution("Promeni
DNS server na 8.8.8.8"));

```

```

when Hypothesis(name == "ip_configuration_problem") NetworkTest(name ==
"adapter_visible", success == true) NetworkAlert(name == "DHCP_FAILED")
then insert(new Hypothesis("dhcp_not_assigning_ip")); insert(new Solution("Pokreni:
ipconfig /release && ipconfig /renew"));

when Hypothesis(name == "ip_configuration_problem") NetworkTest(name ==
"local_loop_test", success == true) NetworkTest(name == "gateway_ping", success == false)
then insert(new Hypothesis("gateway_misconfigured")); insert(new Solution("Proveri
konfiguraciju default gateway-a"));

when Hypothesis(name == "physical_or_ethernet_problem") NetworkTest(name ==
"local_loop_test", success == false)
then insert(new Hypothesis("network_card_malfunction")); insert(new Solution("Proveri
mrežnu karticu u Device Manageru"));

when Hypothesis(name == "physical_or_ethernet_problem") NetworkTest(name ==
"local_loop_test", success == true) NetworkTest(name == "gateway_ping", success == false)
then insert(new Hypothesis("ethernet_cable_disconnected")); insert(new Solution("Proveri
i ponovo poveži Ethernet kabl"));

when Hypothesis(name == "firewall_or_routing_restriction") NetworkTest(name ==
"specific_port_unreachable", success == true) NetworkTest(name == "other_ports_work",
success == true)
then insert(new Hypothesis("firewall_blocking_port")); insert(new Solution("Dodaj firewall
pravilo za određeni port"));

when Hypothesis(name == "bandwidth_or_routing_problem") NetworkAlert(name ==
"PACKET_LOSS_WARNING") NetworkAlert(name == "BANDWIDTH_SATURATED")
then insert(new Hypothesis("isp_bandwidth_limitation")); insert(new Solution("Kontaktiraj
ISP ili nadogradi paket"));

when Hypothesis(name == "dns_or_isp_problem") NetworkAlert(name ==
"PACKET_LOSS_CRITICAL") NetworkTest(name == "other_services_unreachable", success
== true)
then insert(new Hypothesis("isp_outage")); insert(new Solution("Sačekaj ISP ili koristi
mobilni hotspot"));

when Hypothesis(name == "wifi_interference_or_driver") accumulate(PingResult(success
== false); $count : count()) eval($count >= 3)
then insert(new Hypothesis("unstable_wifi_connection")); insert(new Solution("Promeni
WiFi kanal ili ažuriraj driver"));

```

NIVO 3 - KONFLIKT REZOLUCIJA (Final Resolution)

Pravila trećeg nivo-a primenjuju ekspertsko rezonovanje koje iskusan IT inženjer koristi prilikom dijagnostike. Ova pravila kombinuju činjenice generisane iz template-a (NetworkAlert sa pragovima za packet loss, latenciju, DHCP i DNS) sa hipotezama iz prethodnih nivo-a i rezultatima testova kako bi se otkrili kaskadni kvarovi, eliminisale nemoguće hipoteze, prepoznali obrasci i odredio pravi redosled rešavanja problema.

```
when $h : Hypothesis(name == "network_card_malfunction") NetworkTest(name == "local_loop_test", success == true)
```

```
then retract($h);
```

```
when $dns : Hypothesis(name == "dns_server_unreachable") Hypothesis(name == "gateway_misconfigured") NetworkAlert(name == "DNS_SLOW")
```

```
then retract($dns); insert(new Resolution("DNS spor je posledica gateway problema - rešiti samo gateway, DNS će automatski raditi"));
```

```
when NetworkAlert(name == "PACKET_LOSS_HIGH" || name == "PACKET_LOSS_CRITICAL") NetworkAlert(name == "LATENCY_HIGH" || name == "LATENCY_CRITICAL")
```

```
NetworkTest(name == "gateway_ping", success == true)
```

```
then insert(new Resolution("Lokalna mreža radi (gateway OK) ali su packet loss i latencija visoki - problem je kod ISP-a"));
```

```
when NetworkAlert(name == "PACKET_LOSS_CRITICAL") NetworkAlert(name == "LATENCY_CRITICAL") NetworkAlert(name == "DHCP_FAILED")
```

```
then insert(new Resolution("Potpuni kvar mreže - DHCP, packet loss i latencija svi u kritičnom stanju - hitna eskalacija"));
```

```
when NetworkAlert(name == "DNS_SLOW") not NetworkAlert(name == "PACKET_LOSS_HIGH" || name == "PACKET_LOSS_CRITICAL") NetworkTest(name == "gateway_ping", success == true)
```

```
then insert(new Resolution("Izolovan DNS problem bez drugih simptoma - probaj prvo flush DNS cache (ipconfig /flushdns)"));
```

```
when Hypothesis(name == "wifi_interference_or_driver") NetworkAlert(name == "PACKET_LOSS_WARNING" || name == "PACKET_LOSS_HIGH") NetworkTest(name == "wifi_signal_strength_low", success == true)
```

```
then insert(new Resolution("WiFi interferencija - slab signal uz packet loss, promeni WiFi kanal ili pređi bliže routeru"));
```

```

when Hypothesis(name == "wifi_adapter_disabled") Hypothesis(name ==
"dns_server_unreachable")
then insert(new Resolution("Sekvencijalno rešavanje: 1) uključi WiFi adapter, 2) sačekaj
DHCP dodelu IP-a, 3) ponovo testiraj DNS"));

when Hypothesis(name == "isp_outage") NetworkAlert(name ==
"PACKET_LOSS_CRITICAL")
then insert(new Resolution("ISP outage potvrđen template alarmom - prekini dalju
dijagnostiku, čekaj ISP")); insert(new SkipFurtherTesting(true));

when $a : NetworkAlert($n : name) not NetworkAlert(name != $n) not Hypothesis()
NetworkTest(name == "gateway_ping", success == true) NetworkTest(name ==
"dns_server_ping", success == true)
then insert(new Resolution("Izolovan alarm " + $n + " bez drugih indikatora problema -
verovatno trenutna fluktuacija, ponovi merenje kroz minut"));

when Hypothesis(name == "isp_bandwidth_limitation") NetworkAlert(name ==
"PACKET_LOSS_HIGH") NetworkAlert(name == "LATENCY_HIGH") NetworkAlert(name ==
"BANDWIDTH_SATURATED")
then insert(new Resolution("ISP bandwidth limit potvrđen iz tri nezavisna izvora (hipoteza +
3 alarma) - kontaktiraj ISP za upgrade paketa"));

when NetworkAlert(name == "PACKET_LOSS_CRITICAL", $t : timestamp) not
Resolution(status == "resolved") eval(System.currentTimeMillis() - $t > 600000)
then insert(new Resolution("Kritičan packet loss alarm aktivan duže od 10 minuta -
automatska eskalacija na viši nivo podrške"));

```

Accumulate funkcija:

Accumulate funkcija se koristi za agregaciju podataka:

- Broj neuspešnih ping testova u određenom vremenskom periodu
- Detekcija nestabilne veze na osnovu učestalosti grešaka

```

when accumulate(PingFailed() over window:time(1m), $count : count())

    eval($count >= 3)

then insert(new UnstableConnectionDetected($count));

```

Primer Backward Chaining-a:

U sistemu backward chaining se koristi za utvrđivanje da li je određeno stanje mreže ispunjeno, npr. da li korisnik može da pristupi internetu, da li je konekcija stabilna, da li je mreža sigurna.

Sistem polazi od zadatog cilja i deli ga na preduslove koji se dalje granaju do osnovnih činjenica koje se direktno testiraju. Ako neka činjenica ne može da se dokaže, sistem identifikuje tačno na kojoj dubini je problem nastao.

Model

Sistem koristi dve vrste činjenica u radnoj memoriji:

`ServiceDependsOn(service, dependency)` - opisuje da određeni servis zavisi od drugog servisa. Ovakve činjenice se učitavaju iz konfiguracije mrežne topologije i predstavljaju statičko znanje o strukturi sistema.

`ServiceWorks(service)` - direktan dokaz da određeni servis radi, dobijen sprovedenim testom (uspešan ping, ipconfig pokazuje konfiguraciju, status uređaja je aktivan). Ove činjenice se ubacuju dinamički kako Java aplikacija sprovodi mrežne testove.

Kod

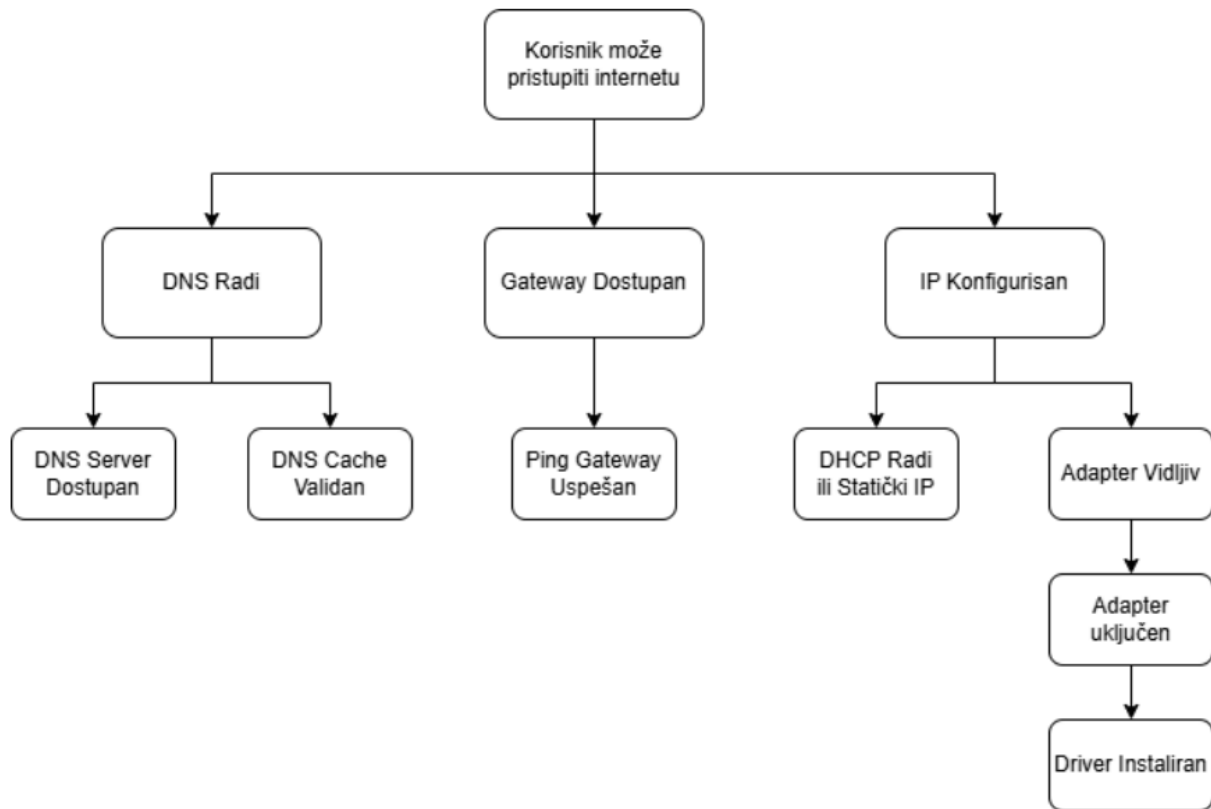
```
query "isAvailable"(String service)
  // BAZNI SLUČAJ: imamo direktan dokaz da servis radi
  ServiceWorks(service == $service := service)

  or

  // REKURZIVNI SLUČAJ: nema direktnog dokaza, ali sve zavisnosti rade
  (
    not ServiceWorks(service == $service := service)
    and ServiceDependsOn(service == $service := service, $dep :
dependency)
    and isAvailable($dep;)
  )
end
```

Primer

Korisnik želi da pristupi internetu. Na slici ispod prikazano je stablo činjenica.



Sistem postavlja cilj „Korisnik može pristupiti internetu" koji zavisi od tri preduslova.

Za svaki preduslov sistem proverava da li je validan ili mora dalje da ga deli. Na primer, „IP konfigurisan" zahteva da DHCP radi, ili da je dodeljen statički IP, i da je adapter vidljiv.

Adapter vidljiv zahteva da je adapter uključen, što zahteva da je driver instaliran.

Ako su sve osnovne činjenice istinite, sistem zaključuje da je cilj dostignut. Ako se neka činjenica ne može dokazati, sistem će identifikovati u čemu je problem.

Template:

Sistem koristi Drools Rule Template mehanizam za dinamičko generisanje pravila iz eksternih podataka. Umesto da se svako pravilo piše ručno u DRL fajlu, template definiše strukturu pravila jednom, a konkretne vrednosti pragova (thresholds) se učitavaju iz eksterne tabele (Excel, CSV, baza podataka) u runtime-u.

U sistemu postoji veliki broj strukturno identičnih pravila koja se razlikuju samo po vrednostima pragova. Na primer, pravila za detekciju nivoa packet loss-a imaju isti oblik, menja se samo numerički prag i naziv činjenice:

- kada je packet_loss > 5% -> PACKET_LOSS_WARNING
- kada je packet_loss > 15% -> PACKET_LOSS_HIGH

- kada je packet_loss > 30% -> PACKET_LOSS_CRITICAL

Bez template-a, svaki novi prag zahteva izmenu DRL fajla i rekompajliranje. Sa template-om, operater može da promeni pragove u Excel tabeli bez dodirivanja koda.

Primer podataka koji bi se učitali iz (iz Excel/CSV tabele):

parameter	operator	threshold	factName	warningMessage	salience
packet_loss	>	5	PACKET_LOSS_WARNING	Packet loss preko 5%	80
packet_loss	>	15	PACKET_LOSS_HIGH	Packet loss preko 15%	90
packet_loss	>	30	PACKET_LOSS_CRITICAL	Packet loss preko 30%	100

parameter	operator	threshold	factName	warningMessage	salience
latency	>	50	LATENCY_WARNING	Latencija preko 50ms	70
latency	>	100	LATENCY_HIGH	Latencija preko 100ms	85
latency	>	200	LATENCY_CRITICAL	Latencija preko 200ms	100

parameter	operator	threshold	factName	warningMessage	salience
dhcp_response_time	>	3000	DHCP_SLOW	DHCP server spor (>3s)	75
dhcp_response_time	>	10000	DHCP_FAILED	DHCP server ne odgovara	95
dns_response_time	>	500	DNS_SLOW	DNS spor (>500ms)	70

Konkretno koje template bi sistem imao:

- Template za detekciju stanja packet loss-a
- Template za detekciju nivoa latencije
- Template za detekciju problema sa DHCP/DNS odgovorom
- Template za detekciju iskorišćenosti propusnog opsega
- Template za detekciju bezbednosnih pretnji (failed logins, port scan attempts)

Napravio bih jedan generički template koji bi primao različite podatke iz Excel/CSV tabele, tako da mogu da generišem različita pravila na osnovu jednog univerzalnog template-a.

Ovo je posebno važno za mrežnu dijagnostiku jer različite mreže imaju različite tolerancije. Na primer, korporativna mreža sa kritičnim aplikacijama može imati strože pragove latencije (>50ms = problem) nego kućna mreža (>200ms = problem), a administrator može da menja te pragove bez intervencije programera. Takođe, pragovi se mogu prilagoditi tipu konekcije (WiFi tolerira veći packet loss od žičane veze) ili dobu dana (drugačiji pragovi tokom peak hours kada je očekivano veće opterećenje).

Zaključak

Expert sistem za mrežnu dijagnostiku omogućava brzu, objašnjivost, i konzistentnu dijagnostiku kroz tri nivoa rezonovanja. Sistem će omogućiti čak i manje iskusnom kadru da rešava mrežne probleme sa visokim procenatom uspešnosti.